ZAINA RAHEEN A.P
241801328

Step 1: Import Required Libraries

```
import pandas as pd
import numpy as np
import seaborn as sns
import matplotlib.pyplot as plt

from sklearn.model_selection import train_test_split
from sklearn.linear_model import LinearRegression
from sklearn.preprocessing import PolynomialFeatures
from sklearn.pipeline import make_pipeline
from sklearn.svm import SVR
from sklearn.metrics import mean_absolute_error, mean_squared_error
from sklearn.cluster import KMeans
from google.colab import files

uploaded = files.upload()
```

```
Choose Files No file chosen  Upload widget is only available when the cell has been executed in the current browser session. Please rerun this cell to enable.
Saving data4.csv to data4.csv
```

Step 2: Load Dataset

```
df = pd.read_csv("data4.csv")

print("First 5 Records")
print(df.head())
```

```
First 5 Records
   area  bedrooms  age  distance    price
0  1200         2    5        10  5000000
1  1500         3    3         8  6500000
2  1800         3    2         5  8000000
3  2000         4    1         4  9500000
4  1100         2   10        15  4500000
```

Step 3: Explore the Dataset

```
print("Dataset Information")
print(df.info())

print("\nStatistical Summary")
print(df.describe())
```

```
Dataset Information
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 10 entries, 0 to 9
Data columns (total 5 columns):
 #   Column    Non-Null Count  Dtype
---  ------    --------------  -----
 0   area      10 non-null     int64
 1   bedrooms  10 non-null     int64
 2   age       10 non-null     int64
 3   distance  10 non-null     int64
 4   price     10 non-null     int64
dtypes: int64(5)
memory usage: 532.0 bytes
None

Statistical Summary
              area   bedrooms        age   distance         price
count    10.000000  10.000000  10.000000  10.000000  1.000000e+01
mean   1700.000000   3.100000   4.200000   7.200000  7.750000e+06
std     439.696865   0.994429   3.047768   4.131182  2.720805e+06
min    1100.000000   2.000000   1.000000   2.000000  4.500000e+06
25%    1425.000000   2.250000   2.000000   4.250000  5.750000e+06
50%    1650.000000   3.000000   3.500000   6.500000  7.250000e+06
75%    1950.000000   3.750000   5.750000   9.500000  9.125000e+06
max    2500.000000   5.000000  10.000000  15.000000  1.300000e+07
```

Step 4: Check Missing Values

```
print(df.isnull().sum())
```

```
area        0
bedrooms    0
age         0
distance    0
price       0
dtype: int64
```

Step 5: Select Features and Target

```
X = df[['area','bedrooms','age','distance']]

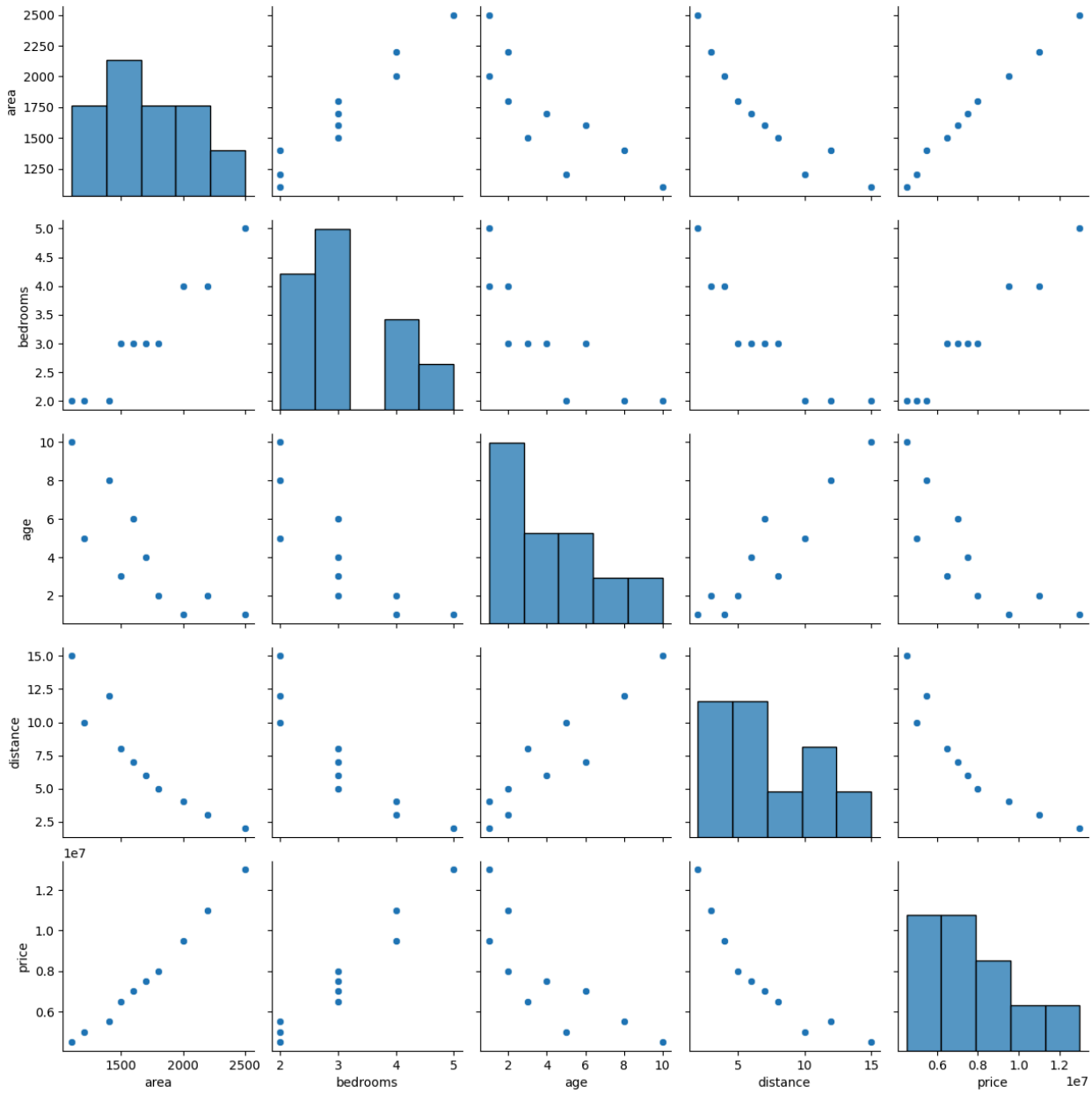
y = df['price']

print(X.head())
print(y.head())
```

```
   area  bedrooms  age  distance
0  1200         2    5        10
1  1500         3    3         8
2  1800         3    2         5
3  2000         4    1         4
4  1100         2   10        15
0    5000000
1    6500000
2    8000000
3    9500000
4    4500000
Name: price, dtype: int64
```

Step 6: Visualize Data

```
sns.pairplot(df)
plt.show()
```



Step 7: Split Dataset

```
X_train, X_test, y_train, y_test = train_test_split(
    X,
    y,
    test_size=0.2,
    random_state=42
)

print("Training Data Shape:", X_train.shape)
print("Testing Data Shape:", X_test.shape)
```

```
Training Data Shape: (8, 4)
Testing Data Shape: (2, 4)
```

Step 8: Linear Regression

```
linear_model = LinearRegression()

linear_model.fit(X_train, y_train)

y_pred_linear = linear_model.predict(X_test)

print(y_pred_linear[:5])
```

```
[12719892.58690936  6846539.49944441]
```

Step 9: Polynomial Regression

```
poly_model = make_pipeline(
    PolynomialFeatures(degree=2),
    LinearRegression()
)

poly_model.fit(X_train, y_train)

y_pred_poly = poly_model.predict(X_test)

print(y_pred_poly[:5])
```

```
[13630863.07528789  6347926.86486178]
```

Step 10: Support Vector Regression (SVR)

```
svr_model = SVR(kernel='rbf')

svr_model.fit(X_train, y_train)

y_pred_svr = svr_model.predict(X_test)

print(y_pred_svr[:5])
```

```
[7250001.24060797 7249999.6892347 ]
[7250001.24060797 7249999.6892347 ]
```

Step 11: Evaluate Linear Regression

```
mae = mean_absolute_error(y_test, y_pred_linear)
mse = mean_squared_error(y_test, y_pred_linear)
rmse = np.sqrt(mse)

print("Linear Regression")
print("MAE :", mae)
print("MSE :", mse)
print("RMSE:", rmse)
```

```
Linear Regression
MAE : 313323.45626752824
MSE : 99274893771.75842
RMSE: 315079.18650992867
```

Step 12: Evaluate Polynomial Regression

```
mae = mean_absolute_error(y_test, y_pred_poly)
mse = mean_squared_error(y_test, y_pred_poly)
rmse = np.sqrt(mse)

print("Polynomial Regression")
print("MAE :", mae)
print("MSE :", mse)
print("RMSE:", rmse)
```

```
Polynomial Regression
MAE : 391468.10521305446
MSE : 210557229096.22943
RMSE: 458865.15349961957
```

Step 13: Evaluate SVR

```
mae = mean_absolute_error(y_test, y_pred_svr)
mse = mean_squared_error(y_test, y_pred_svr)
rmse = np.sqrt(mse)

print("SVR")
print("MAE :", mae)
print("MSE :", mse)
print("RMSE:", rmse)
```

```
SVR
MAE : 3249999.2243133653
MSE : 16812492633431.008
RMSE: 4100303.968418806
```

Step 14: Apply K-Means Clustering

```
kmeans = KMeans(
    n_clusters=3,
    random_state=42,
    n_init=10
)

kmeans.fit(X)

df['Cluster'] = kmeans.labels_

print(df[['area','bedrooms','age','distance','Cluster']].head())
```

```
   area  bedrooms  age  distance  Cluster
0  1200         2    5        10        0
1  1500         3    3         8        2
2  1800         3    2         5        2
3  2000         4    1         4        1
4  1100         2   10        15        0
```

Step 15: Visualize Clusters

```
sns.scatterplot(
    data=df,
    x='area',
    y='price',
    hue='Cluster'
)

plt.title("House Price Clusters")
plt.show()
```